Function

➢ **Syntax**

```
function [msb:lsb]
function_name;

input [msb:lsb]
input_arguments ;

reg [msb:lsb] reg_variable_list;

parameter [msb:lsb]
parameter_list;

integer [msb:lsb] integer_list;

begin
  statements ...
end

endfunction
```

Usage

- Functions are used to describe combinational logic.
- Functions are used when a piece of code is repeated many times. They are used to make code re-usable.

Declaration

- **Functions** must be declared within a module it called in.
- **Functions** begins with keyword function and ends with keyword endfunction
- **Functions** can take multiple inputs and returns only a single output (function name).
- regs, parameters and integer declarations is allowed.
- **Function** core is a single statement, or a sequence of statements surrounded by begin ... End
- NO timing controls as (#, @(posedge),<=,...) is allowed

© 2022 Eng. Ali Eltemsah

Function

```
module simple_function (
  input  [7:0] A, B ,
  output [3:0] A_ones, B_ones);

assign A_ones = OnesCount (A) ;
assign B_ones = OnesCount (B) ;

function [3:0] OnesCount;
input [7:0] X;
integer I;

begin
OnesCount = 0;
   for (I = 0; I <= 7; I = I + 1)
    if (X[I]) OnesCount = OnesCount + 1;
end
endfunction

endmodule
```

Function Call

- Functions can be called from **continuous assignments**, **always blocks** or **other functions**.
- Functions are **synthesizable**.

Function can't declare outputs or inout signals

Function Name is used as reg variable

Functions must contain a statement that assigns the return value to the implicit function name register

Task

➢ **Syntax**

```
task task_name;
input [msb:lsb]
input_arguments ;
output [msb:lsb]
output_arguments ;
reg [msb:lsb] reg_variable_list;
parameter [msb:lsb]
parameter_list;
integer [msb:lsb] integer_list;
begin
  statements ...
end

endtask
```

Usage

- Tasks are used to describe combinational logic.
- Tasks are used when a piece of code is repeated many times. They are used to make code re-usable.

Declaration

- **Task** begins with keyword task and ends with keyword endtask
- **Task can take multiple or zero inputs and returns multiple or zero outputs**
- regs, parameters and integer declarations is allowed.
- **Task** core is a single statement, or a sequence of statements surrounded by begin ... End
- **Timing controls as (#, @(posedge),<=,...) is allowed**

© 2022 Eng. Ali Eltemsah

Task

➢ **Style 1**

```
task task_name;
input [port_list];
output [port_list];
parameter
[parameter_list];
integer integer_list;

begin
  statements ...
end

endtask
```

➢ **Style 2**

```
task task_name (
input [In_list],
output [out_list]
);
parameter [parameter_list];
integer [msb:lsb] integer_list;

begin
  statements ...
end

endtask
```

➢ **Style 3** (Empty port list)

```
task task_name ();
parameter [parameter_list];
integer [msb:lsb]
integer_list;

begin
  statements ...
end

endtask
```

© 2022 Eng. Ali Eltemsah

For Loop

- For loops can be used in both **synthesizable** and **non-synthesizable** code

- For loops in synthesizable code are mainly used to:-
  1. Initialize 2D array (Memories) with reset values
  2. Expand replicated logic. They are simply a way of shrinking the amount of code that is written by the hardware designer.

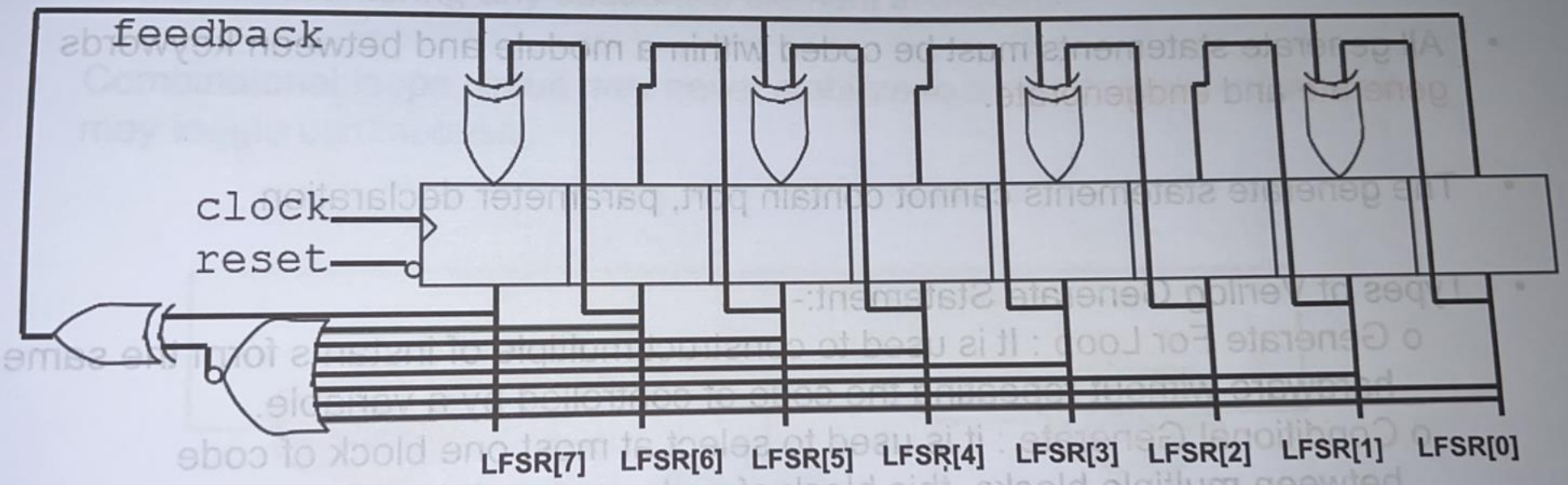
- Syntax

```
for ( <initial_condition> ; condition ; <step assignment> )
  begin
    statement
  end
```

© 2022 Eng. Ali Eltemsah

For Loop – Example

LFSR (Linear Feedback Shift Register)



© 2022 Eng. Ali Eltemsah

Verilog generate block

- Verilog generate statement is a powerful construct for writing configurable, reusable and synthesizable RTL. It provide the ability for the design to be generalized as it is built based on Verilog parameters.

- All generate statements must be coded within a module and between keywords generate and endgenerate.

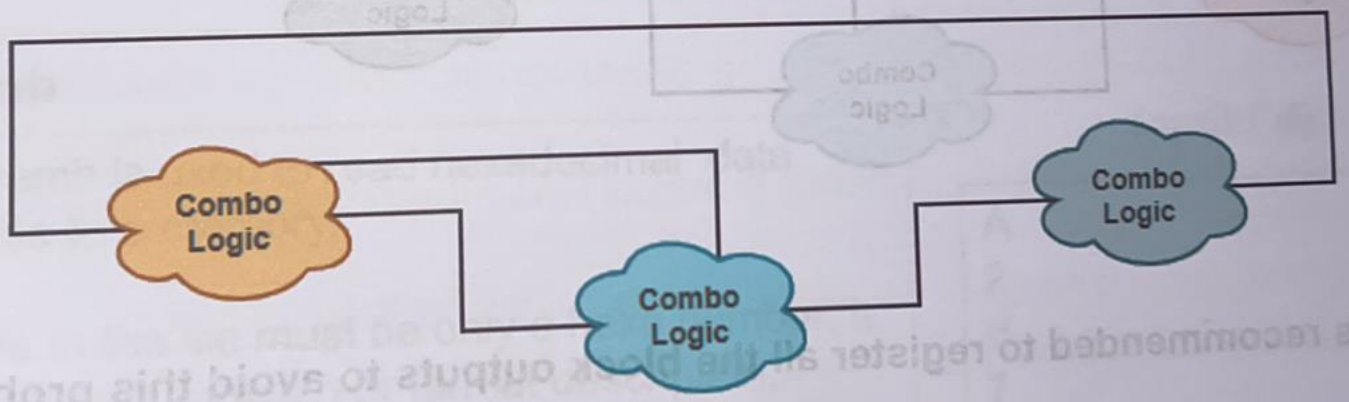
- The generate statements cannot contain port, parameter declaration.

- Types of Verilog Generate Statement:-
  - o Generate For Loop : It is used to construct multiple of instants form the same hardware without repeating the code of controlled by a variable.
  - o Conditional Generate : it is used to select at most one block of code between multiple blocks, this block of code can contain always/initial blocks or assign statements. Conditional generate constructs include two forms:-
    - -- if generate
    - -- casegenerate.

Combinational Loop

- A combinational **loop** is structure which is formed by a signal starting from an input of a **combinational** gate, after passing through one or more **combinational** gate, reaches the same combo gate from which it started without encountering any sequential element in between.

- Combinatorial loops circuit may never stabilize to a steady state value, as it may toggle continuously.

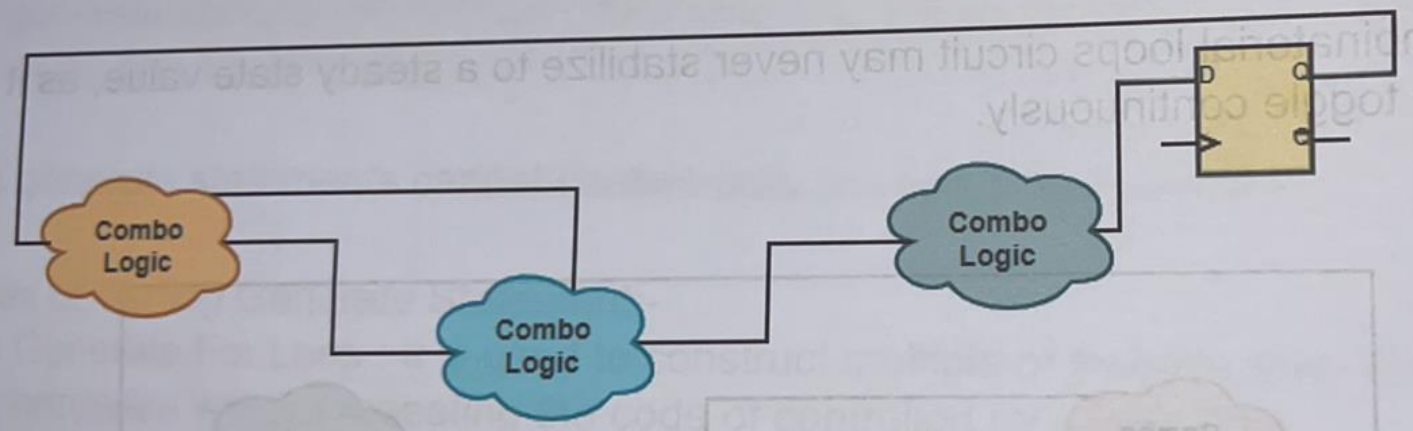


© 2022 Eng. Ali Eltemsah

Combinational Loop

- How to Fix it ?

>> By adding a register to break the feedback loop



Note: it is recommended to register all the block outputs to avoid this problem

© 2022 Eng. Ali Eltemsah

Testbench Input Data File

$readmemb

- $readmemb is used to read binary data from a file to a memory.
- Numbers in the file must be only a binary number, it cannot contain bit width or format description, numbers can be separated by spaces or new line.
- Syntax : $readmemb("binary_file.txt", Mem);

Input File

```
1010
0010
1101
0111
```

$readmemh

- $readmemh is used to read hexadecimal data from a file to a memory.
- Numbers in the file must be only a hexa number, it cannot contain bit width or format description, numbers can be separated by spaces or new line.
- Syntax : $readmemh("hexadec_file.txt", Mem);

Input File

```
A
2
d
7
```

© 2022 Eng. Ali Eltemsah

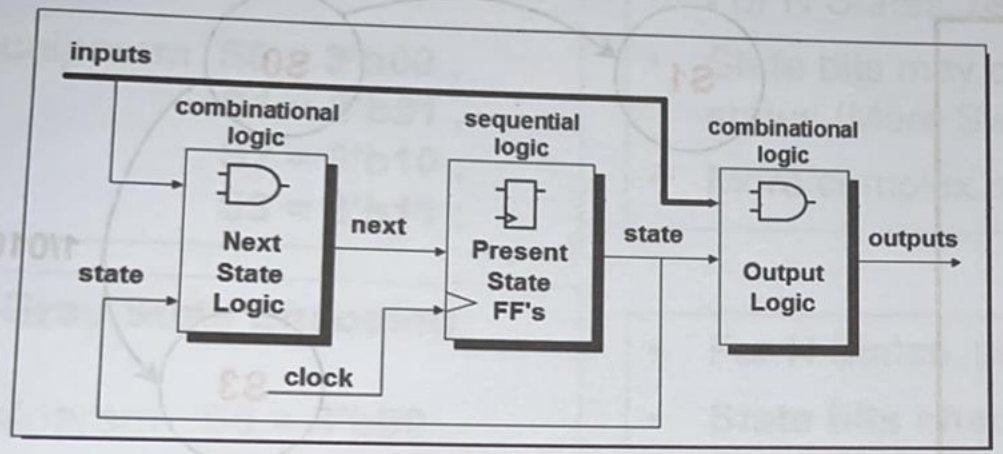
Introduction To FSM

- Finite state machine (FSM) is a very useful abstraction model for digital circuit design that perform a predetermined sequence of actions depending on events.
- Finite state machine (FSM) is a major tool to control sequential operations.
- FSM has finite inputs, outputs and number of states. At each clock edge, combinational logic computes outputs and next state as function of inputs and present state
- The behavior of state machines can be observed in many devices in modern society such as :-
  - **Vending Machine:** which dispense products when the proper combination of coins is deposited
  - **Elevators:** whose sequence of stops is determined by the floors requested by riders.
  - **Traffic Light:** which change color in sequence when cars are waiting.

© 2022 Eng. Ali Eltemsah

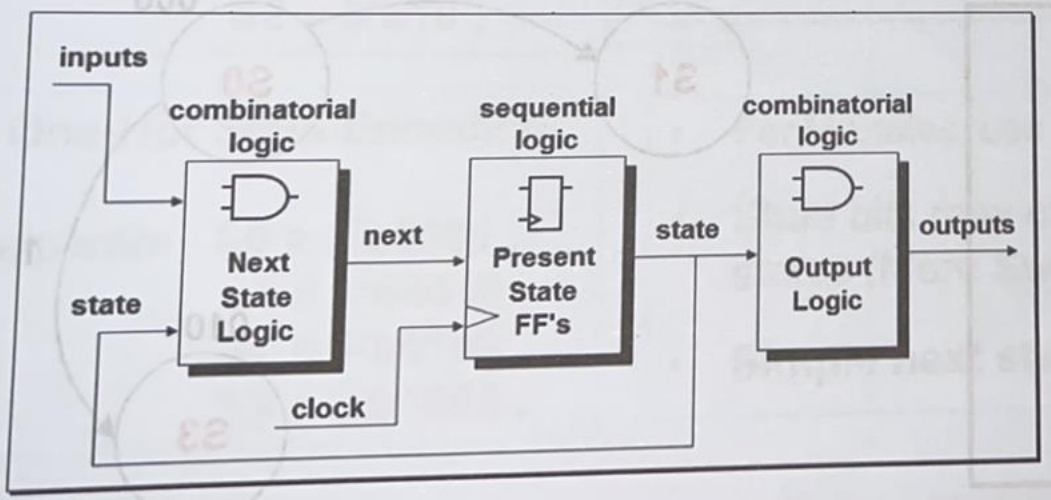
Types of Finite State Machines

- **Mealy State Machine**



- Mealy Outputs depends on both the present state and the inputs.

- **Moore State Machine**

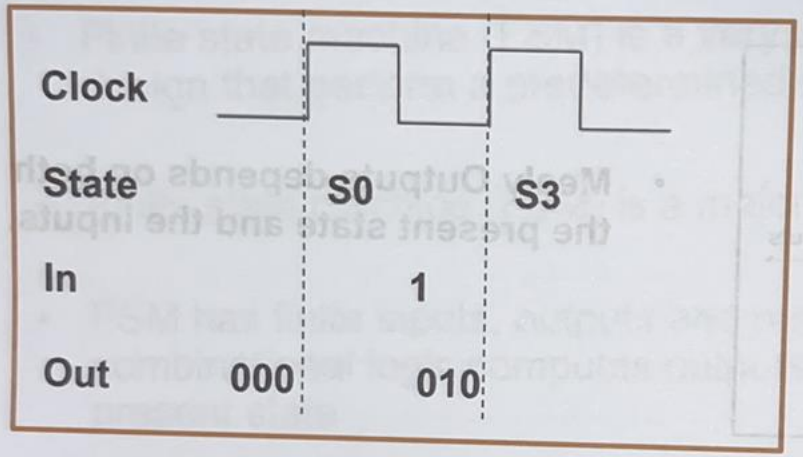


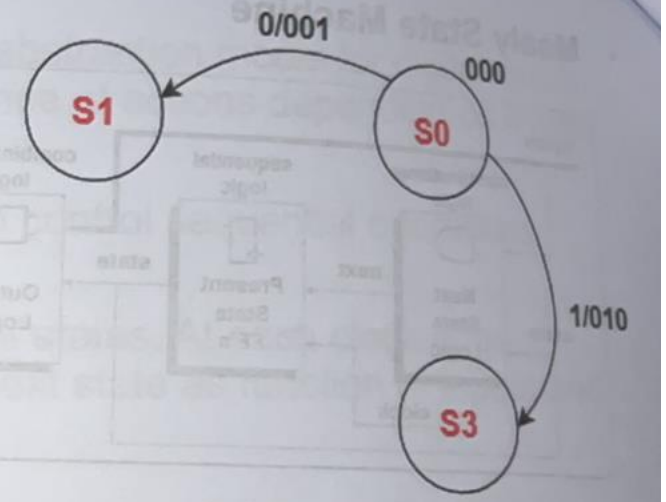
- Moore Outputs depends only on the present state.

© 2022 Eng. Ali Eltemsah

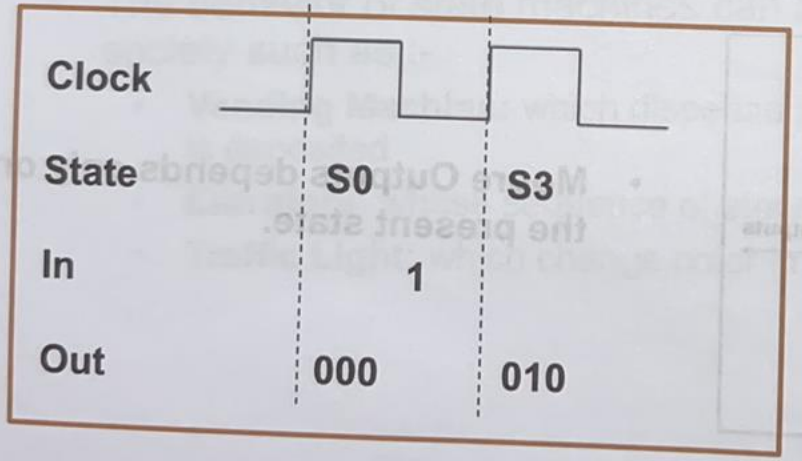
Types of Finite State Machines

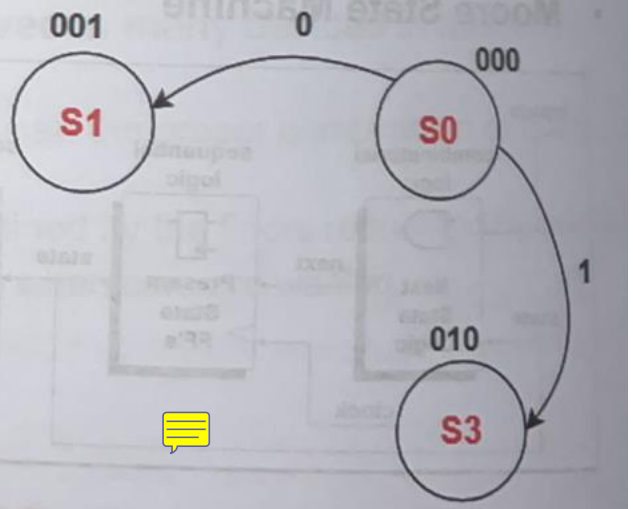
- **Mealy State Machine**





- **Moore State Machine**





© 2022 Eng. Ali Eltemsah

Finite State Encoding

- **Binary State Encoding**

```
Localparam  S0 = 3'b00 ,
            S1 = 3'b01 ,
            S2 = 3'b10 ,
            S3 = 3'b11 ;
```

- For N States, use ceil($log_2 N$) registers to encode the states.
- State bits may change more than 1 bit between sequential states (More Switching Power)
- More complex next state combinational logic (slow)

- **Gray State Encoding**

```
localparam  S0 = 3'b00 ,
            S1 = 3'b01 ,
            S2 = 3'b11 ,
            S3 = 3'b10 ;
```

- For N States, use ceil($log_2 N$) registers to encode the states.
- **State bits change only 1 bit between sequential states (Less Switching Power)**
- More complex next state combinational logic (slow)

- **One-Hot State Encoding**

```
localparam  S0 = 3'b0001 ,
            S1 = 3'b0010 ,
            S2 = 3'b0100 ,
            S3 = 3'b1000 ;
```

- For N States, use N registers to encode states **(More Hardware).**
- State bits may change more than 1 bit between sequential states (More Switching Power)
- **Simple next state combinational logic (Fastest)**

© 2022 Eng. Ali Eltemsah

FSM Coding Styles

Style 1 :Three Always Block FSM

- FSM Encoding
- State Transition (sequential always)
- Next state logic (Comb always)
- Output logic (Comb always)

Style 2 :Two Always Block FSM

- FSM Encoding
- State Transition (sequential always)
- Next state logic & Output logic (Comb always)

© 2022 Eng. Ali Eltemsah

Let's Practice

GOAL:

Build an electronic combination lock with a reset button, two number buttons (0 and 1), and an unlock output. The combination should be 01011.

STEPS:

1. Design lock FSM (block diagram, state transitions)
2. Write Verilog module(s) for FSM

© 2022 Eng. Ali Eltemsah

Delay Based Timing Control

Inter-Assignment Delays

- **#<delay> <LHS> = <RHS>**
- **Inter-assignment delay** statement has delay value on the **left-hand side** of the assignment operator. This indicates that both **evaluation** and **assignment** of the statement is done ***after*** the delay expires.
- **#5 A = B ;**

Intra-Assignment Delays

- **<LHS> = #<delay> <RHS>**
- **Intra-assignment delay** statement has delay value on the **right-hand side** of the assignment operator. This indicates that **evaluation** of the statement on the RHS is done first then the **assignment** of the result to the LHS variable is done ***after*** the delay expires. .
- **A = #5 B ;**

© 2022 Eng. Ali Eltemsah

Event Based Timing Control

@ (posedge signal) | **@ (negedge signal)**

- **@ (posedge …)** event controls the execution of a following statements on the signal transition from 0 to X, Z or 1 and **@ (negedge …)** event controls the execution of a following statements on the signal transition from 1 to X, Z or 0.
- It block the execution of the following statements until it is executed.
- It is **edge sensitive** based timing control.
- **Ex : @(posedge valid) in = 3'b011** | **@(negedge clock) in = 3'b011;**

wait (condition)

- The **wait statement** evaluates a condition,
  - if it is true, the procedural statements will be executed.
  - if it is false, the procedural statements remain blocked until the condition becomes true.
- It block the execution of the following statements until it is executed.
- It is **Level sensitive** based timing control.
- **Ex : wait (Out_Valid)**

© 2022 Eng. Ali Eltemsah

Forever & Repeat

Forever Loop

- The forever loop executes continually, the loop never ends.
- forever statements normally used inside initial blocks to avoid simulation hanging.
- Syntax : forever <statement> ;
- Ex:-

```
initial
  begin
    forever #10 clk = ~clk ;
  end
```

Repeat

- The repeat loop executes <statement> a fixed <number> of times
- syntax : repeat (<number>) <statement>
- Ex: **repeat(10) @(posedge clk);** //wait 10 clks

© 2022 Eng. Ali Eltemsah